Foundations of Systems Security

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-18

Foundations: Lecture Overview

The Course: what this unit is, what it is not, how it is assessed, and the machine you will use

Security Properties: security's duality, CIA and beyond, policy against mechanism, reference monitors

Adversaries and Trust: the attacker, the ladder of positions, trust boundaries, least privilege, layering

Systems Security, Specifically: what is left once network and cryptography are somebody else's problem

Into the Lab: the four dimensions of a local attack surface, and what you enumerate today

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

Computing Systems Security

- Semester 1 of the MCS, and open to MEI through the Cryptography and Information Security profile
- 5 ECTS. The accredited allocation is 45 contact hours, one third lecture and two thirds laboratory
- **Vítor Fonte**: lectures and course coordination
- **José Diogo Martins**: every laboratory class, and shared grading
- Thirteen weeks here, one lecture hour and one two-hour laboratory each: 39 of those 45 contact hours
- Materials are in English. Ask questions in whichever language you prefer

The laboratory is where this unit is learned. The lecture hour exists to make the laboratory hour make sense.

What This Unit Is Not

- **Not network security.** The wire, its protocols and its attacks are left out on purpose
- **Not cryptography.** Our Week 4 levels what a signature proves; the mathematics belongs to a cryptography unit
- **Not software security.** Memory corruption and injection belong to Software Security in Semester 2
- **Not penetration testing.** Reconnaissance, scanning and methodology are not taught here
- What is left is the machine itself: users, processes, files, privilege, and the boundaries between them

Systems security is what remains once the network and the cryptography are someone else's problem.

The Semester in Five Blocks

- **Block 1, Weeks 1 and 2.** Threat modelling and attack surface: what is here to defend, and against whom
- **Block 2, Weeks 3 to 5.** Authentication and authorisation: PAM, federated identity, SSH in depth
- **Block 3, Weeks 6 to 8.** The Unix privilege model, capabilities, mandatory access control
- **Block 4, Weeks 9 to 11.** Isolation in depth: namespaces, cgroups and seccomp, container runtimes
- **Block 5, Weeks 12 and 13.** The attacker's perspective: escape and escalation chains, then the capstone defence

Blocks 2 to 4 each install a defence on your machine. Block 5 takes them away and asks whether you notice.

How You Are Assessed

- **Individual written test, 40 per cent.** Artefact-grounded: you read output and configuration, not definitions
- **Individual laboratory reports, 20 per cent.** Four across the semester, one per block, written alone
- **Group project with individual assessment, 40 per cent.** Groups of three, defended orally in Week 13
- The oral defence moves each student's mark away from the group mark, up or down
- Weights sit inside the ranges the accredited unit description fixes; they are not negotiable mid-semester

Minimum 8 out of 20 on each of the three components. Below that on any one of them fails the unit, whatever the weighted average says.

The Machine You Will Work On

```
$ grep -E 'version|arch' /etc/ssc-image-release
version: 1.0
built: cloud-init single-source, dual-arch native
arch: arm64

$ docker run --rm hello-world
Hello from Docker!
```

- Apple Silicon: `ssc-lab-arm64-v1.0.1.utm`
- Intel or AMD: `ssc-lab-amd64-v1.0.ova`
- User and password `student`: run `passwd`
- Work from your own terminal: `ssh -p 2222 student@localhost`
- **No shared folder.** This machine runs exploits

Snapshot before every laboratory, restore after. Take today's snapshot yourself and name it lab1-clean.

Lab Core

Working Conventions

- Groups of three for the project, formed in Week 2 and fixed for the semester
- Laboratory reports are individual, even though the bench work is done in the same groups of three
- Bring the machine to every laboratory. Sessions are hands-on from the first minute
- The capstone is released in Week 10: the machine you have been hardening, with its defences removed
- One attacker model, all semester: an unprivileged, non-owner foothold, never the machine's owner

Every defence you install between Weeks 5 and 12 is a link in the chain you will be asked to break in Week 13.

Security Properties

What exactly are we protecting, from whom, and for whose benefit?

The Duality of Security

- Nothing is secure in the abstract. Secure **of what**, **against whom**, **for what purpose**
- A door lock is neither secure nor insecure: adequate against a passer-by, useless against a locksmith
- Change any one of the three and the answer changes, with no change whatsoever to the system
- The same file permission is a real control against a local account and irrelevant against somebody holding the disk
- This is why every mechanism in this unit is introduced with the adversary it is meant to stop

Any claim of the form this system is secure is incomplete until it names the property and the adversary.

CIA, and What It Leaves Out

- **Confidentiality.** A configuration file holds a database password. Which accounts on this machine can read it?
- **Integrity.** A script runs as root on a schedule. Which accounts can change what it does before it next runs?
- **Availability.** A service answers on a port. Who can make it stop answering?
- **Authenticity.** Is this really the user, the host, the container image it claims to be?
- **Accountability.** Can you say afterwards who did it, and from where?
- **Non-repudiation.** Can they credibly deny having done it?

Every one of these six questions has a concrete answer on the machine in front of you. Today you go and find them.

Assets, Actors, Motivations, Threats

- **Asset.** Something worth protecting: data, credentials, compute, and the machine's standing as a trusted peer
- **Actor.** Anyone who can act on the system. Named by position and capability, not by intent alone
- **Motivation.** Why they bother: money, a route onwards, disruption, curiosity
- **Threat.** A specific bad outcome that an actor with a motivation could cause to an asset
- A vulnerability is none of these. It is the weakness that lets a threat become an incident

These four nouns are the columns of the threat model you build in Week 2. Today's inventory is the evidence you will build them from.

Policy and Mechanism

- A **policy** says what is allowed: only the report service may read its own credentials
- A **mechanism** enforces it: an ownership and a mode, a dedicated service account, a unit-file directive
- A policy can be wrong while perfectly enforced; a mechanism can be right while enforcing nothing
- Most of this unit is mechanisms. Blocks 2 to 4 are nine weeks of them
- Block 1 is where policy is set, and the only block whose right answer depends on what you decide to value

Ask of any control: is this the policy or the mechanism? Systems fail at the seam between them.

The Reference Monitor

- Anderson, 1972: every access to every object passes through one component that consults the policy
- **Complete mediation.** No path around it. If one code path skips the check, the monitor is decoration
- **Tamper-resistance.** The monitor cannot be modified by the things it mediates
- **Verifiability.** Small enough that you can convince yourself it is correct
- No real system achieves all three. The Linux kernel mediates completely and is far too large to verify
- Every mechanism in this unit approximates the ideal, and its failures are one of the three properties breaking

When a defence fails, ask which of the three broke. More often than not it is complete mediation.

References: Anderson 1972

Adversaries and Trust

If you cannot name the attacker, what exactly is your defence for?

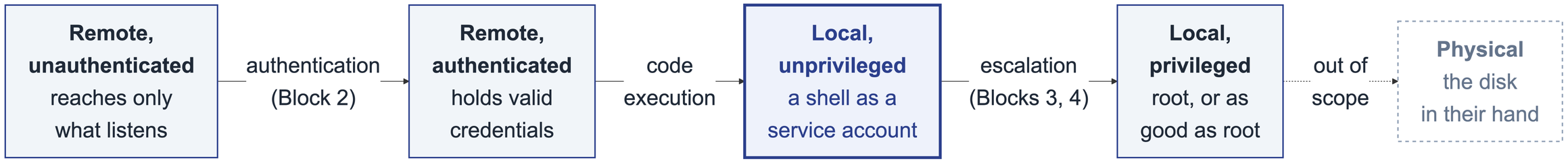
Name the Adversary

- **Position.** Off the machine, on it as a user, in a service account, or holding the disk
- **Capabilities.** What they can do from there, and how much time, money and noise they will spend doing it
- **Goals.** What counts as a win. Often not the obvious asset but a foothold towards something else
- A model with no stated attacker cannot be tested, because every claim it makes is unfalsifiable
- It is also how you decide what **not** to defend: a defence with no adversary is cost with no benefit

This unit's adversary, all semester: an unprivileged, non-owner foothold on your machine. Never the owner.

A Ladder of Adversaries for One Host

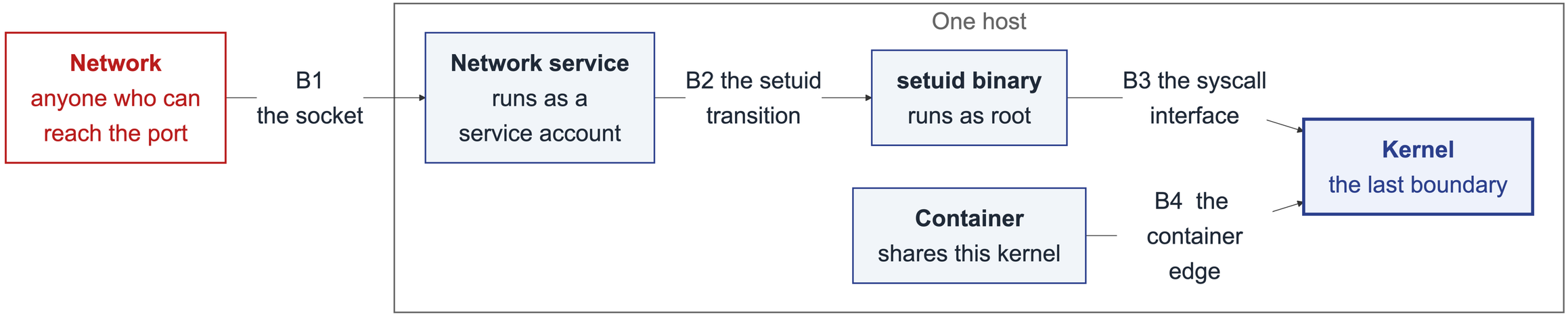
- Each rung is a **position**, and each arrow is the **boundary** that separates it from the next
- The interesting engineering question is never the rung but the arrow: what has to hold for the attacker not to climb
- This unit lives on the third rung and spends nine weeks making the third arrow expensive
- Physical access is honestly out of scope: disk encryption is a real answer, and it is not this unit's answer



Name the rung you are defending. A defence that does not say which arrow it holds cannot be assessed.

Trust Boundaries

- A trust boundary is a line where data or control passes between parts that trust differently
- Every boundary is where a check has to happen, and every unchecked crossing is a waiting bug
- The dangerous ones are the boundaries nobody drew, such as a root-run script anyone may edit
- Drawing these lines is the whole content of next week's data-flow diagram



Four boundaries, four weeks that exist to teach them: the socket, the setuid transition, the syscall interface, the container edge.

Least Privilege and Privilege Separation

- **Least privilege.** Every component holds exactly the rights it needs, and no more. Saltzer and Schroeder, 1975
- **Privilege separation.** Split the program so the privileged part is as small as possible
- Principles today, mechanisms later: a service account, a scoped sudo rule, a dropped capability
- The hard part is never stating them. It is finding out which rights are actually needed, which is an empirical question
- The default fails both. A program running as root holds every right, and one bug hands over all of them

You will apply least privilege as a measurement, not a slogan: drop, run, observe what breaks, repeat.

References: Saltzer and Schroeder 1975

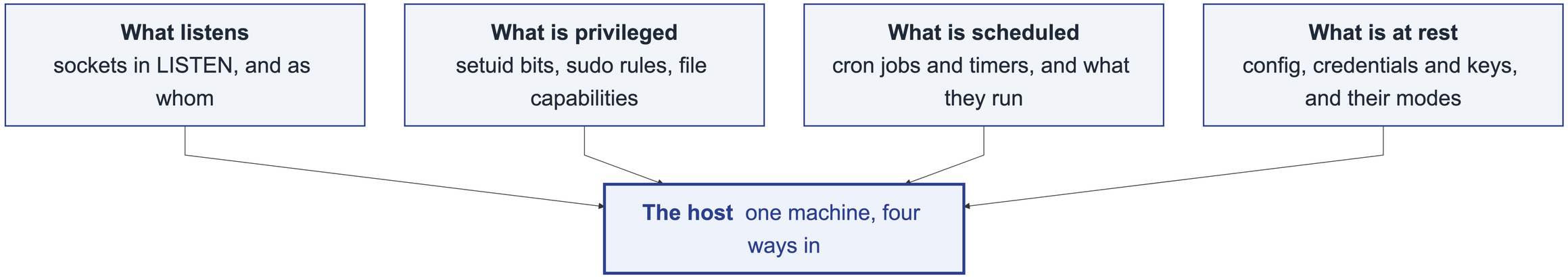
Defence in Depth

- No single boundary holds. Assume each will fail, and ask what the next one still prevents
- Layering beats perfecting one layer, because the failure modes are usually independent
- An escape needs a namespace gap **and** a capability **and** an allowed syscall **and** a permitting policy
- Measured on both course images: grant `CAP_SYS_ADMIN` and the refusal does not disappear, it **moves** from the syscall filter to mandatory access control. Only when both are lowered does the mount succeed
- That is also why `--privileged` is dangerous: it does not add one power, it removes every layer

Every layer removed is a layer that was silently doing work nobody had to think about.

The Attack Surface: Four Dimensions

- The attack surface is the sum of the points where an adversary can interact with the system
- On a single host these four dimensions cover nearly all of it
- The dimensions matter more than any list. They turn look for problems into four answerable questions
- Answering all four, for one machine, is exactly what you do in the next two hours
- The four are not exhaustive: kernel modules, firmware and the hypervisor sit outside them



These four are the structure of today's laboratory, of next week's threat model, and of the capstone's target.

Attack Surface as a Quantity

Post-session reading. The metric itself is developed in the Week 2 lecture.

- Counting is crude but not useless: seven listeners is worse than two, other things being equal
- Weighting is where it becomes a model. A socket bound to `0.0.0.0` counts for more than one on `127.0.0.1`
- A usable metric is **monotone**: removing an item never increases it. That is what makes it a measure
- What it cannot do is rank two different systems, or tell you which single remaining item is the fatal one
- So use it as a delta, on one machine, across one change. Never as an absolute score

This is the number that shrinks every week from Week 5 onward, and jumps straight back up in Week 13.

Study Support

Systems Security, Specifically

The network is somebody else's problem and the cryptography is a given. What is actually left?

Attack surface so far: four dimensions named, none of them measured yet. Measurement starts in today's laboratory

What Is Left

- Take away the wire and the mathematics, and what remains is the machine's internal trust structure
- **Who** a process is: a set of identities, not a person. Real, effective and saved, and the capability sets behind them
- **What** it may touch: files and their modes, other processes, devices, and the kernel's syscall surface
- **How** it changes identity: the setuid transition, sudo, PAM, a user namespace
- **What watches**: audit records and denial logs, and whether anybody ever reads them
- That is the whole syllabus, and all of it is reachable from a shell prompt on your own machine

Cryptography protects data in transit and at rest. In use it is decrypted, and only the operating system stands between it and the next process.

The Unix Model in One Slide

```
$ id -un ; id -nG
student
student sudo docker

$ stat -c '%A %U %G %n' /etc/shadow
-rw-r----- root shadow /etc/shadow
```

- Everything is a file, or is reached through something that behaves like one
- Every process carries an identity, and the kernel decides from that identity, not from the program
- Access is three bits, three times: owner, group, everyone else. Coarse, fast, and everywhere
- Privilege is largely **one bit**: uid 0 either can do everything or is an ordinary account
- Capabilities exist because that bit was too blunt, and splitting it was harder than it looked

A first sketch, deliberately imprecise. Block 3 makes every line of it exact.

Where Systems Security Actually Fails

- The exotic exploit is the memorable case. It is not the common one
- A service left running that nobody needs, on a machine nobody owns any more
- A permission widened once, for a deadline, and never narrowed afterwards
- A credential in a file that is world-readable because nobody thought about the default
- A scheduled job running a script from a directory somebody else can write to
- An account created for a migration years ago that still has a shell and a sudo rule

None of these needs a CVE, an exploit, or skill. They need somebody to have looked, and nobody did.

Observation Before Judgement

A different machine, not your lab VM. Observation first, judgement second, kept apart.

```
$ systemctl cat db-backup.service | grep Exec
ExecStart=/srv/jobs/db.sh
$ systemctl show -p User db-backup.service
User=root

$ ls -l /srv ; ls -l /srv/jobs
drwxrwxr-x 2 root staff 4096 Sep  8 09:02 jobs
-rwxr-xr-x 1 root root   412 Sep  8 09:02 db.sh
```

- **Observation.** A service the machine starts on its own runs `/srv/jobs/db.sh` as `root`
- **Observation.** The script itself is writable only by root. Its **directory** is writable by group `staff`
- **Judgement, separately.** Whoever can write the directory chooses what root runs
- Note what is **not** claimed: that anyone unprivileged is on this machine at all
- Collapsing the two steps produces confident findings that do not survive the first question

Lab Core

Today's Laboratory

```
ss -tlnp             # what listens, and as whom
find / -perm -4000 -type f 2>/dev/null
sudo -l              # what is privileged
cat /etc/cron.d/*    # what is scheduled
ls -l /etc/*/*.conf # what is at rest
```

- Arrive with the machine installed (Week 0); leave with it snapshotted as `lab1-clean`
- For every listener, record the address, the program, **and the account behind it**
- Most of what you find is standard issue. Whatever is not, record it and leave it alone
- The handout's Part F is one table, four rows, and a column for the question you cannot yet answer
- Your own root access is **out of scope**. The adversary is the service account, not you

No exploitation today. The single skill is enumeration, and the deliverable is an inventory.

Lab Core

What to Bring to Week 2

- The inventory table from Part F: four rows, and one open question against each
- Your open questions are next week's agenda. Week 2 turns them into a threat model with actors and attack trees
- Laboratory report 1, due 11 October: the model, the inventory, Labs 1 and 2 preparation answers
- From Week 3 to Week 12 you install one defence at a time on that same machine
- In Week 13 you get the same machine with those defences removed, and your job is to walk in

By December you will have built a machine's defences week by week and then taken them apart. Today you write down what is there.

Reading and Reference

- **Stallings and Brown, Computer Security: Principles and Practice, 4th ed., 2018**. The unit's general reference. Chapter 1 for today's properties and vocabulary.
- **Pfleeger, Pfleeger and Coles-Kemp, Security in Computing, 6th ed., 2023**. An alternative treatment of the same foundations, often clearer on policy.
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. The closest book to this unit's laboratory style. Chapter 1 for set-UID.
- **Saltzer and Schroeder, The Protection of Information in Computer Systems, 1975**. Where least privilege comes from, with seven others still in use.
  https://www.cs.virginia.edu/~evans/cs551/saltzer/
- **Anderson, Computer Security Technology Planning Study, 1972**. Where the reference monitor comes from.
  https://csrc.nist.gov/publications/history/ande72.pdf

Reference

Key Takeaways

- Security is always of something, against someone, for a purpose. A claim missing any of the three is not yet a claim
- A policy says what is allowed; a mechanism enforces it. Nearly every incident lives at that seam
- Name the adversary or the defence cannot be assessed. Ours is an unprivileged, non-owner foothold
- The local attack surface has four dimensions: what listens, what is privileged, what is scheduled, what is at rest
- Enumerate before you judge. The inventory is today's deliverable; the judgement comes next week

Defence in depth, measured on both course images: the capability granted, the filter switched off, and the operation still refused.